



Anjuman-I-Islam's

M. H. SABOO SIDDIK COLLEGE OF ENGINEERING

8, Saboo Saddik Polytechnic Road, Byculla, Mumbai, Maharashtra 400008

DEPARTMENT OF INFORMATION TECHNOLOGY

Mini Project ITL605 DS LAB

Title of the Project

Drowsy Driver Detection

Supervisor/Guide

PROF.FARZANA KHAN

REV - 2019 'C' Scheme



University of Mumbai

Academic Year (2024 -25)

GROUP MEMBERS

STUDENT NAME	ENROLLMENT NO.
YAMIN KHAN	242464
ABDURRAHMAN QURESHI	242466
SHAIKH RUMAAN	242469
SHAIKH SAMEER	242471

TABLE OF CONTENT

Sr.No	Content	Page No
1	Problem Definition	1
2	System Architecture	2
3	Tech Stack	3
4	Results And Visualizations	4
5	Conclusion or Business Insights	7

PROBLEM DEFINITION

Driver drowsiness is a major cause of road accidents worldwide. Fatigue reduces reaction time, awareness, and decision-making ability, which can lead to serious or fatal crashes. Traditional methods like self-awareness or manual monitoring are unreliable.

The problem is to **develop a real-time system that can automatically detect whether a driver is active, drowsy, or sleeping** using a webcam and alert them before an accident occurs.

This system aims to:

- Monitor the driver continuously
- Detect eye closure and blinking patterns
- Classify driver state into **Active, Drowsy, or Sleeping**
- Provide visual feedback (and can be extended to alarms)

SYSTEM ARCHITECTURE

The system follows a **computer vision pipeline**:

Step-by-step Flow:

1. Input Layer

- Webcam captures real-time video frames using OpenCV

2. Preprocessing

- Convert frame to grayscale for faster processing

3. Face Detection

- Use `dlib.get_frontal_face_detector()` to detect face region

4. Facial Landmark Detection

- Use pre-trained model `shape_predictor_68_face_landmarks.dat`
- Extract 68 facial points (eyes, nose, mouth, etc.)

5. Eye Detection & Feature Extraction

- Focus on eye landmarks (points 36–47)
- Compute distances between vertical and horizontal eye points

6. Eye Aspect Ratio (EAR) Calculation

- EAR determines whether the eye is open or closed
- Based on ratio:
 - High → Eye open
 - Medium → Drowsy
 - Low → Closed

7. State Classification Logic

- Counters used:
 - sleep
 - drowsy
 - active
- Based on thresholds:
 - **Active :)**
 - **Drowsy !**
 - **SLEEPING !!!**

8. Output Layer

- Display status on screen using OpenCV
- Show:
 - Face bounding box
 - Landmark points
 - Driver state text

TECH STACK

Programming Language :

- Python

Libraries Used :

1. **OpenCV (cv2)**
 - Video capture
 - Image processing
 - Display output
2. **NumPy (numpy)**
 - Mathematical operations (distance calculation)
3. **dlib**
 - Face detection
 - Facial landmark detection
4. **imutils**
 - Simplifies landmark handling (face_utils)

Model Used :

- shape_predictor_68_face_landmarks.dat
 - Pre-trained facial landmark detection model

Hardware Requirements

- Webcam
- Computer/Laptop (real-time processing)

RESULTS AND VISUALIZATION

Code :

```
import cv2
import numpy as np
import dlib
from imutils import face_utils

cap = cv2.VideoCapture(0)
detector = dlib.get_frontal_face_detector()
predictor = dlib.shape_predictor("shape_predictor_68_face_landmarks.dat")
sleep = 0
drowsy = 0
active = 0
status = ""
color = (0, 0, 0)
face_frame = None

def compute(ptA, ptB):
    dist = np.linalg.norm(ptA - ptB)
    return dist

def blinked(a, b, c, d, e, f):
    up = compute(b, d) + compute(c, e)
    down = compute(a, f)
    ratio = up / (2.0 * down)
    if ratio > 0.25:
        return 2
    elif ratio > 0.21 and ratio <= 0.25:
        return 1
    else:
        return 0

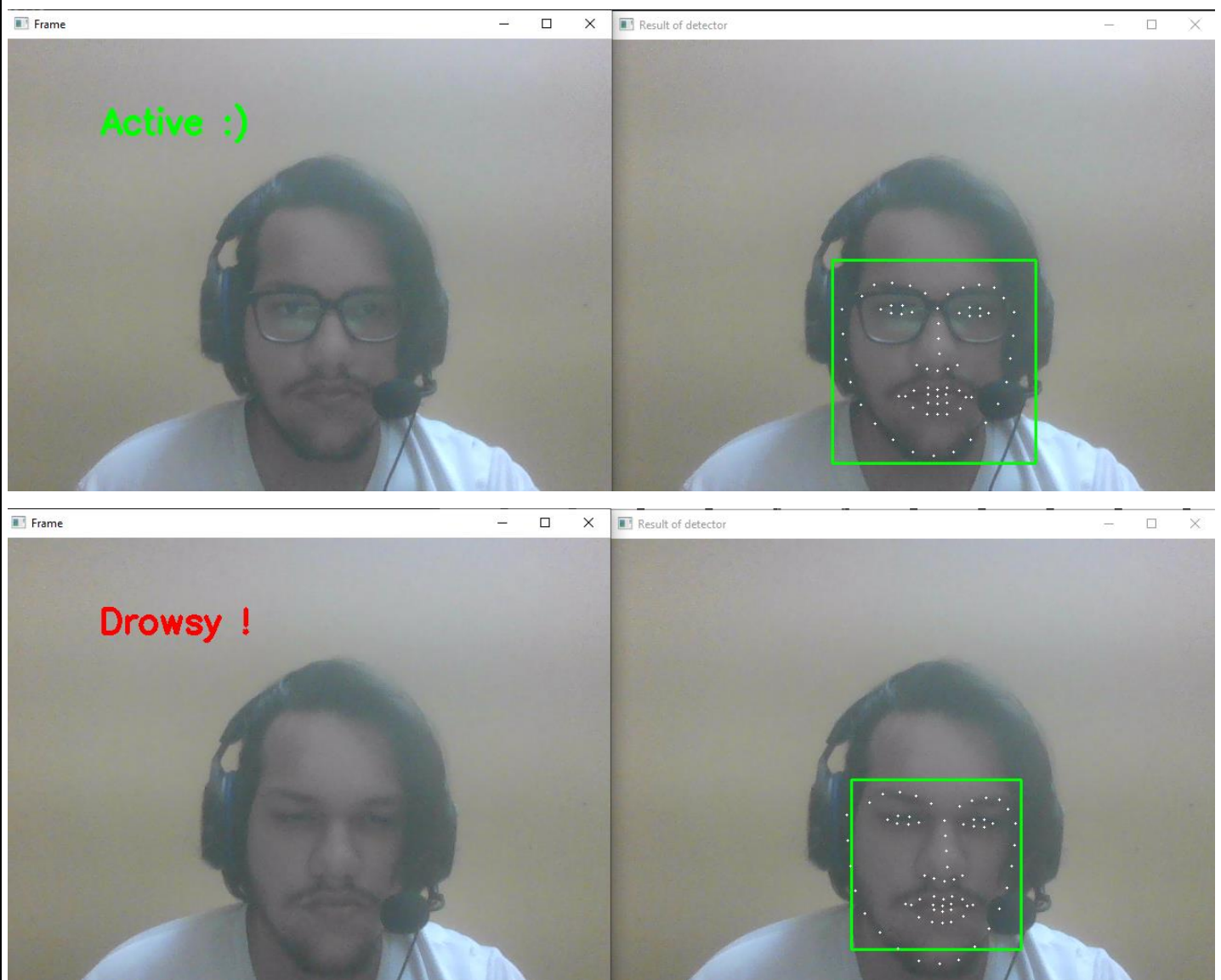
while True:
    _, frame = cap.read()
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = detector(gray)
    face_frame = frame.copy()
    for face in faces:
        x1 = face.left()
        y1 = face.top()
        x2 = face.right()
        y2 = face.bottom()
        cv2.rectangle(face_frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
        landmarks = predictor(gray, face)
        landmarks = face_utils.shape_to_np(landmarks)
        left_blink = blinked(landmarks[36], landmarks[37],
                             landmarks[38], landmarks[41], landmarks[40], landmarks[39])
        right_blink = blinked(landmarks[42], landmarks[43],
                              landmarks[44], landmarks[47], landmarks[46], landmarks[45])
        if left_blink == 0 or right_blink == 0:
            sleep += 1
            drowsy = 0
            active = 0
            if sleep > 6:
                status = "SLEEPING !!!"
                color = (255, 0, 0)
        elif left_blink == 1 or right_blink == 1:
            sleep = 0
            active = 0
            drowsy += 1
            if drowsy > 6:
                status = "Drowsy !"
                color = (0, 0, 255)
        else:
            drowsy = 0
            sleep = 0
```

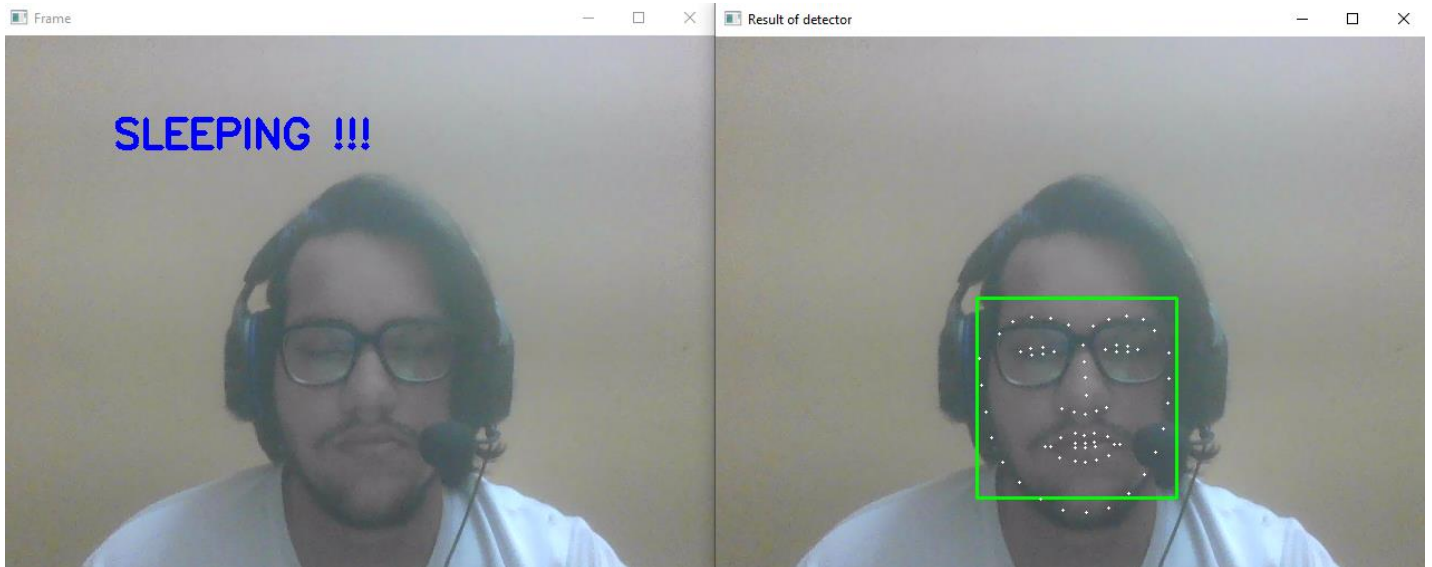
```

    active += 1
    if active > 6:
        status = "Active :)"
        color = (0, 255, 0)
    cv2.putText(frame, status, (100, 100),
                cv2.FONT_HERSHEY_SIMPLEX, 1.2, color, 3)
    for n in range(0, 68):
        (x, y) = landmarks[n]
        cv2.circle(face_frame, (x, y), 1, (255, 255, 255), -1)
cv2.imshow("Frame", frame)
if face_frame is not None:
    cv2.imshow("Result of detector", face_frame)
key = cv2.waitKey(1)
if key == 27:
    break

```

Output :





CONCLUSION

The Driver Drowsiness Detection System successfully uses **computer vision and facial landmark analysis** to monitor driver alertness in real time.

Key achievements:

- Detects eye blinking and closure accurately
- Classifies driver state into **Active, Drowsy, and Sleeping**
- Works in real-time using a simple webcam
- Provides visual feedback for safety

Future Improvements

- Add **alarm/buzzer system** when drowsy detected
- Integrate with **IoT (Arduino/ESP32)** for vehicle systems
- Improve accuracy using **Deep Learning (CNN/LSTM)**
- Deploy as a **mobile** or **embedded system**